Machine Learning System Design interview
Ad Click Prediction
Introduction and Overview
Online advertising system's objective is to find and place their advertisements ads to a platform for measurable responses such as impressions, clicks, conversions. Displaying relevant ads boosts engagement and revenue for ad-hosting entities platforms such as Google, Facebook, and Instagram.
Clarifying Requirements
Here's a typical interaction between a candidate and an interviewer.
Candidate: Can I assume the business objective of ad prediction system is to maximize revenue?
Interviewer: Yes, that's correct.
Candidate: There are different types of ads, such as video and image ads. In addition, ads can be displayed in different sizes and formats, like square, landscape, pop-up ads, etc. For simplicity, can I assume ads are placed on users' timelines only, and every click generates the same revenue?
Interviewer: That sounds good.
Candidate: Does the system show the same ad to different users?
Interviewer: No, it shows personalized ads. The system should be able to show ads multiple impressions to eligible consumers (users) in the past. Ad fatigue (that is, the point where the users are fed up with the ad) is important. If the user repeatedly sees ads, it has negatively affected ad-free subscriber's experience.
Candidate: Do we suppose that "more like ad" feedback mean about "hide this advertisement"? These kinds of negative feedback help us to detect irrelevant ads.
Interviewer: Good point. Suppose the system also considers users' dislikes as well as their likes. "Hide this advertisement" is an interesting feature, but we don't need to consider it for now.
Candidate: Would it be also fair to assume that the training dataset should be constructed using user and ad data, and the labels derived from user ad interactions?
Interviewer: Yes, that's correct.
Candidate: Since we can construct positive training data points via user clicks, how do we generate negative data points? Can we assume any impressions with no clicks are negative data points? What if the user scrolls fast and doesn't spend time seeing the ad? What if an ad was an impression on negative, but eventually, the user clicks on it?
Interviewer: These are excellent questions. Where do you propose?
Candidate: If an ad is hidden or a user is known to have a certain dislike for an ad, we can count it as a negative data point. In addition, we can also consider a dwell time threshold. For example, if a user spends less than 2 seconds on an ad, we can consider it as a negative feedback and not a false click. We can also use a dwell time threshold to construct the training dataset.
Interviewer: Dwell time is an interesting idea. Suppose we do use a dwell time threshold to create our training dataset. Can you think of any drawbacks of using a dwell time threshold?
Candidate: Users can be distracted while viewing an ad. For example, they may open a new tab or switch to another app. In such cases, the dwell time may be misleading. We can also consider other complex techniques to deal with such cases, but for now, we can keep it simple and use a dwell time threshold.
Interviewer: Sounds good. Let's move on.
Frame the Problem as an ML Task
Defining the ML objective
The goal of the ad click prediction system is to increase revenue by showing users ads they are more likely to click on. This can be converted into the following ML objective: predicting P(ad | user, context). This is the probability that the user will click on the ad. For example, if the model's output is 0.4, we can say that there is a 40% chance that the user will click on the ad. Figure 1 shows the model's input and output.
Figure 1: Ad click prediction system's input and output.
Choosing the right ML category
Figure 2 illustrates the ad prediction system as framed as a ranking problem. As described in Chapter 2 Event Recommendations System, the potential candidate's ranking is ideal for a ranking system that optimizes for ranking relevancy problems. The pointwise LTR employs a binary classification model that takes a user, an ad, and a potential candidate as input and predicts the probability that user will click on the ad. Figure 2 shows the model's input and output.
Figure 2: Binary classification model's input/ output.
Data Preparation
Data engineering
Here's some raw data available in this system.
Ads
Users and interactions.
Ads
An Ads is chosen to fit 3 criteria. In practice, we may have hundreds of attributes associated with each ad. For simplicity, we only list important ones.
Ad ID	Advertiser ID	Ad group ID	Campaign ID	Category	Subcategory	Images or Videos
1	1	4	5	travel	flight	[link, display-name] [,...]
2	2	6	7	Electronics	Cellphone	[link, display-name] [,...]
3	1	4	5	travel	airline	[link, display-name] [,...]
Table 1: Ads
Users
The scheme for user data is stored below.
User ID	Name	Gender	City	Country	Language	Time zone
1	Alex	M	Seattle	USA	English	-7
Table 2: Scheme for user data
User ad interactions
This table stores user ad interactions such as impressions, clicks, and conversions.
User ID	Ad ID	Interaction type	Dwell time	Location (for ads)	Timestamp
1	1	impression	null	Kirkland, Washington, USA	1624455600
10	2	impression	5.41 sec	JF-2001, JECS-000	1624455 0800
4	20	Click	-	DU-7001, JF-0802	1624455 0800
15	8	Conversion	-	DU-7001, JF-0802	1624455 0800
Table 3.1: User ad interaction data
Feature engineering
Our aim in this section is to engineer features that will assist us in predicting user clicks.
Ad features
Ad features include the following.
Ids
Categorizations
Images and videos
Impression and click numbers
Let's examine each in more detail.
Ids
These are different Ids: campaign Id, ad group Id, ad Id, etc.
Why is it important? The Ids represent the advertiser, the campaign, the ad group, and the ad itself. These Ids are used as predictive features to provide their particular attributes of different advertisers, campaigns, ad groups, and ads.
How to prepare it? We embedding layer converts sparse features, such as IDs, into dense feature vectors. Each ID type has its own embedding layer.
Categorizations
Why is it important? A video or image is a point in space in another signal that can help us predict whether that ad will be clicked on by a user. For example, a video ad may be more likely to be clicked on than an image ad.
How to prepare it? The images or videos are first preprocessed. After that, we use a pre-trained model such as SimCLR [1] to convert preprocessed images into dense vectors.
Ad category and subcategory
As provided by the advertiser, this is the ad's category and subcategory. For example, here is a list of broad types of categories that are targetable: Arts & Entertainment, Autos & Vehicles, Beauty & Fitness, etc.
Why is it important? It helps the model understand which category the advertiser is targeting.
How to prepare it? Categorizations are also supplied by an advertiser. Therefore, we can use an embedding layer to learn them.
Why is it important? These numbers indicate how often users reacted to this ad. For example, users are more likely to click on an ad with a high click through rate (CTR).
Figure 3.1: Ad feature preparation for user embeddings and interactions.
User features
Similar to previous chapters, we choose the following features.
Demographics: age, gender, city, country, etc.
Contextual information: device, time of the day, etc.
Interactions between users: clicks, conversions, user's historical engagement statistics, etc.
Let's take a closer look at interaction-related features.
Clicked ads
Ads previously clicked by the user.
Why is it important? This indicates a user's interests. For example, when a user clicks on lots of insurance-related ads, it suggests they are likely to click on a similar ad again.
How to prepare it? We can use a sliding window to collect the "Ad features".
User's historical engagement statistics
These are the user's historical engagement numbers, such as their total ad views and ad click rate.
Why is it important? An individual's statistics represent an overall profile of a typical user's engagement. In general, users are more likely to click on ads from a user's profile that they have clicked on frequently in the past.
How to prepare it? Engagement statistics are represented as numerical values. To prepare them, we scale their values into a similar range.
Figure 3.2: Feature preparation for user embeddings and interactions.
Notice concluding the data preparation section, let's examine a common challenge in ad click prediction systems. In most cases, the number of ads is much smaller than the number of users. Therefore, the ad feature may not be sufficiently dense to provide enough predictive signals. Similarly, historical interactions between users and ads are sparse. Since the huge feature space (which others and other values), it is common to have thousands of features or more. To provide more predictive signals, in the model selection section, we will cover techniques to overcome these unique challenges.
Model Development
Model selection
As described in the system "Frame the Problem as an ML Task", the binary classification model is chosen to solve the ranking problem. Binary classifications can be modeled in several different ways. The following are common choices in ad click prediction systems.
Logistic regression
Factorization machines
Tree-based models
Gradient boosted decision trees / logistic regression
Neural networks
Tree-based models
Factorization machines
Deep Factorization Machines
Logistic regression [2,3]
LR models the probability of a binary outcome using a linear combination of one or multiple features. LR is fast to train and easy to implement and deploy. Its simplicity and speed make it a popular choice for many ad click prediction systems.
Non-linear problems can not be modeled with LR. LR solves this task using a linear combination of input features, which leads to underfitting. To address this, we can add non-linear features to the input feature set, such as feature crosses and bucketization.
Inability to capture feature interactions. LR is not capable of capturing feature interactions. In ad prediction systems, it is very common to have various interactions between features. When feature interact with each other, the output probability depends on the combination of the two features, not just the effect of one feature depends on the value of the other feature.
Feature crossing. Feature crossing can help us add non-linear features and capture feature interactions. However, because it is fast to implement and easy to train, many companies use it to create a baseline model.
Feature crossing v LR
To capture feature interactions better, we can add feature crosses to the input feature set.
What is feature crossing?
Feature crossing is a technique used in ML to create new features from existing features. It involves combining two or more features to capture the interaction between them. The idea is that by combining features, the model can learn more complex interactions between the original features in the data, which can improve the performance of ML models. For example, interactions such as "young and basketball" or "USA and football" may positively impact a model's ability to predict clicks probability.
How to create feature crosses?
In feature crossing, we manually add new features to the existing features based on prior knowledge. As Figure 4 shows, common feature crosses can be "age and city" or "language" pairs, as these features are meaningfully dependent. To learn more about crossing, refer to [4].
Figure 4: Creating feature crosses from two categorical features.
How to use feature crossing in LR?
As shown in Figure 5.2, feature crossing + LR works as follows:
Use feature crossing on the original set of features to extract new features (crossed features).
Use the original and the crossed features as input for the LR model.
Figure 5: Feature crossing.
Figure 5.1: Feature crossing + LR.
This method allows the model to capture complex and non-linear relationships between features. It has three disadvantages.
Manual process: Human involvement is required to choose features to be crossed, which is time-consuming and expensive.
Requires domain knowledge: Feature crossing requires domain expertise. To determine which interactions between features are meaningful enough for the model, we need to understand the problem and the feature space in detail.
Limited to linear models: Feature crossed features can not capture non-linear relationships. To address these issues, we can use factorization machines (FM).
Factorization machines (FM)
FMs automatically learn feature interactions by representing each feature with a low-dimensional latent vector. FMs model feature interactions by calculating the dot product of their embeddings. Let's look at the equation and understand it step-by-step.
Figure 6.1: Simple embedding.
Figure 6.2: Simple embedding.
Two latent architectures: In this option, we use two encoders: user encoder and ad encoder. The similarity between ad and user embeddings is used to determine the relevancy. This is a two-tower model, which is covered in Chapter 2 Event Recommendations.
Figure 6.3: Two-tower architecture.
Although Wide Kaiss works beautifully, they may not be the best choice for ad click prediction systems because:
Sparsity: Given that the feature space is usually huge and sparse, wide linear models and Wide&Deep models can still memorize sparse feature interactions, but they can't generalize to unseen feature combinations.
Difficult to capture all pairwise feature interactions: due to the huge number of features.
Given these limitations, we will use DeepFM.
Deep & Cross Network (DCN)
Deep & Cross Network (DCN) was proposed by Google researchers in 2017 to address the feature interaction sparsity. This addresses the challenges of the manual feature crossing technique. The architecture is shown below. It consists of two components:
Deep network: Learns complex and generalizable features using a Deep Neural Network (DNN) architecture.
Cross network: Automatically creates explicit feature interactions and learns their importance.
The output of deep network and cross network are concatenated to make a final prediction.
There are two types of DCN architectures: stacked and parallel. Figure 8.2 shows the architecture of parallel DCN. To learn more about stacked architectures, refer to [7]. Note that it is not usually expected to provide details of DCN during ML system design interviews. If you are interested in learning more about DCN variations, refer to [7, 8].
Figure 8.1: Simple embedding.
Figure 8.2: DCN architecture.
Deep Factorization Machines (DeepFM)
DCN is a promising approach that combines explicit feature interactions learned from the cross network module. However, the cross network only models certain feature interactions, which may negatively affect the performance of the cross network module.
Factorization Machines (FM)
FMs are promising approaches that combine explicit feature interactions learned from the cross network module. However, the cross network only models certain feature interactions, which may negatively affect the performance of the cross network module.
Factorization Machines (FM)
FMs automatically learn feature interactions by representing each feature with a low-dimensional latent vector. FMs model feature interactions by calculating the dot product of their embeddings. Let's look at the equation and understand it step-by-step.
Figure 8.3: DCN architecture.

08
Ad Click Prediction on Social Platforms

Introduction

Online advertising allows advertisers to bid and place their advertisements ads to a platform for measurable responses such as impressions, clicks, and conversions. Displaying relevant ads boosts engagement and revenue for ad-hosting entities platforms such as Google, Facebook, and Instagram.

Figure 0.1: Sponsored ads placed on the user's timeline.

In this chapter, we design an ad click prediction system. Let's begin similar to other popular social media platforms use.

Clarifying Requirements

Here's a typical interaction between a candidate and an interviewer.

Candidate: Can I assume the business objective of ad prediction system is to maximize revenue?

Interviewer: Yes, that's correct.

Candidate: There are different types of ads, such as video and image ads. In addition, ads can be displayed in different sizes and formats, like square, landscape, pop-up ads, etc. For simplicity, can I assume ads are placed on users' timelines only, and every click generates the same revenue?

Interviewer: That sounds good.

Candidate: Does the system show the same ad to different users?

Interviewer: No, it shows personalized ads. The system should be able to show ads multiple impressions to eligible consumers (users) in the past. Ad fatigue (that is, the point where the users are fed up with the ad) is important. If the user repeatedly sees ads, it has negatively affected ad-free subscriber's experience.

Candidate: Do we suppose that "more like ad" feedback mean about "hide this advertisement"? These kinds of negative feedback help us to detect irrelevant ads.

Interviewer: Good point. Suppose the system also considers users' dislikes as well as their likes. "Hide this advertisement" is an interesting feature, but we don't need to consider it for now.

Candidate: Would it be also fair to assume that the training dataset should be constructed using user and ad data, and the labels derived from user ad interactions?

Interviewer: Yes, that's correct.

Candidate: Since we can construct positive training data points via user clicks, how do we generate negative data points? Can we assume any impressions with no clicks are negative data points? What if the user scrolls fast and doesn't spend time seeing the ad? What if an ad was an impression on negative, but eventually, the user clicks on it?

Interviewer: These are excellent questions. Where do you propose?

Candidate: If an ad is hidden or a user is known to have a certain dislike for an ad, we can count it as a negative data point. In addition, we can also consider a dwell time threshold. For example, if a user spends less than 2 seconds on an ad, we can consider it as a negative feedback and not a false click. We can also use a dwell time threshold to construct the training dataset.

Interviewer: Dwell time is an interesting idea. Suppose we do use a dwell time threshold to create our training dataset. Can you think of any drawbacks of using a dwell time threshold?

Candidate: Users can be distracted while viewing an ad. For example, they may open a new tab or switch to another app. In such cases, the dwell time may be misleading. We can also consider other complex techniques to deal with such cases, but for now, we can keep it simple and use a dwell time threshold.

Interviewer: Sounds good. Let's move on.

Frame the Problem as an ML Task

Defining the ML objective

The goal of the ad click prediction system is to increase revenue by showing users ads they are more likely to click on. This can be converted into the following ML objective: predicting P(ad | user, context). This is the probability that the user will click on the ad. For example, if the model's output is 0.4, we can say that there is a 40% chance that the user will click on the ad. Figure 1 shows the model's input and output.

Figure 1: Ad click prediction system's input and output.

Choosing the right ML category

Figure 2 illustrates the ad prediction system as framed as a ranking problem. As described in Chapter 2 Event Recommendations System, the pointwise LTR employs a binary classification model that takes a user, an ad, and a potential candidate as input and predicts the probability that user will click on the ad. Figure 2 shows the model's input and output.

Figure 2: Binary classification model's input/ output.

Data Preparation

Data engineering

Here's some raw data available in this system.

- Ads
- Users
- User ad interactions

Ads

An Ads is chosen to fit 3 criteria. In practice, we may have hundreds of attributes associated with each ad. For simplicity, we only list important ones.

Ad ID	Advertiser ID	Ad group ID	Campaign ID	Category	Subcategory	Images or Videos
1	1	4	5	travel	flight	[link, display-name] [,...]
2	2	6	7	Electronics	Cellphone	[link, display-name] [,...]
3	1	4	5	travel	airline	[link, display-name] [,...]

Table 1: Ads

Users

The scheme for user data is stored below.

User ID	Name	Gender	City	Country	Language	Time zone
1	Alex	M	Seattle	USA	English	-7

Table 2: Scheme for user data

User ad interactions

This table stores user ad interactions such as impressions, clicks, and conversions.

User ID	Ad ID	Interaction type	Dwell time	Location (for ads)	Timestamp
1	1	impression	null	Kirkland, Washington, USA	1624455600
10	2	impression	5.41 sec	JF-2001, JECS-000	1624455 0800
4	20	Click	-	DU-7001, JF-0802	1624455 0800
15	8	Conversion	-	DU-7001, JF-0802	1624455 0800

Table 3.1: User ad interaction data

Feature engineering

Our aim in this section is to engineer features that will assist us in predicting user clicks.

Ad features

Ad features include the following.

- Ids
- Categorizations
- Images and videos/gifs
- Impression and click numbers

Let's examine each in more detail.

Ids

These are different Ids: campaign Id, ad group Id, ad Id, etc.

Why is it important? The Ids represent the advertiser, the campaign, the ad group, and the ad itself. These Ids are used as predictive features to provide their particular attributes of different advertisers, campaigns, ad groups, and ads.

How to prepare it? We embedding layer converts sparse features, such as IDs, into dense feature vectors. Each ID type has its own embedding layer.

Categorizations

Why is it important? A video or image is a point in another signal that can help us predict whether that ad will be clicked on by a user. For example, a video ad may be more likely to be clicked on than an image ad.

How to prepare it? The images or videos are first preprocessed. After that, we use a pre-trained model such as SimCLR [1] to convert preprocessed images into dense vectors.

Ad category and subcategory

As provided by the advertiser, this is the ad's category and subcategory. For example, here is a list of broad types of categories that are targetable: Arts & Entertainment, Autos & Vehicles, Beauty & Fitness, etc.

Why is it important? It helps the model understand which category the advertiser is targeting.

How to prepare it? Categorizations are also supplied by an advertiser. Therefore, we can use an embedding layer to learn them.

Impression and click numbers

Why is it important? These numbers indicate how often users reacted to this ad. For example, users are more likely to click on an ad with a high click through rate (CTR).

Figure 3.1: Ad feature preparation for user embeddings and interactions.

User features

Similar to previous chapters, we choose the following features.

- Demographics: age, gender, city, country, etc.
- Contextual information: device, time of the day, etc.
- Interactions between users: clicks, conversions, user's historical engagement statistics, etc.

Let's take a closer look at interaction-related features.

Clicked ads

Ads previously clicked by the user.

Why is it important? This indicates a user's interests. For example, when a user clicks on lots of insurance-related ads, it suggests they are likely to click on a similar ad again.

How to prepare it? We can use a sliding window to collect the "Ad features".

User's historical engagement statistics

These are the user's historical engagement numbers, such as their total ad views and ad click rate.

Why is it important? An individual's statistics represent an overall profile of a typical user's engagement. In general, users are more likely to click on ads from a user's profile that they have clicked on frequently in the past.

How to prepare it? Engagement statistics are represented as numerical values. To prepare them, we scale their values into a similar range.

Figure 3.2: Feature preparation for user embeddings and interactions.

Notice concluding the data preparation section, let's examine a common challenge in ad click prediction systems. In most cases, the number of ads is much smaller than the number of users. Therefore, the ad feature may not be sufficiently dense to provide enough predictive signals. Similarly, historical interactions between users and ads are sparse. Since the huge feature space (which others and other values), it is common to have thousands of features or more. To provide more predictive signals, in the model selection section, we will cover techniques to overcome these unique challenges.

Model Development

Model selection

As described in the system "Frame the Problem as an ML Task", the binary classification model is chosen to solve the ranking problem. Binary classifications can be modeled in several different ways. The following are common choices in ad click prediction systems.

- Logistic regression
- Factorization machines
- Tree-based models
- Gradient boosted decision trees / logistic regression
- Neural networks
- Tree-based models
- Factorization machines
- Deep Factorization Machines

Logistic regression [2,3]

LR models the probability of a binary outcome using a linear combination of one or multiple features. LR is fast to train and easy to implement and deploy. Its simplicity and speed make it a popular choice for many ad click prediction systems.

- **Non-linear problems can not be modeled with LR.** LR solves this task using a linear combination of input features, which leads to underfitting. To address this, we can add non-linear features to the input feature set, such as feature crosses and bucketization.
- **Inability to capture feature interactions.** LR is not capable of capturing feature interactions. In ad prediction systems, it is very common to have various interactions between features. When feature interact with each other, the output probability depends on the combination of the two features, not just the effect of one feature depends on the value of the other feature.
- **Feature crossing.** Feature crossing can help us add non-linear features and capture feature interactions. However, because it is fast to implement and easy to train, many companies use it to create a baseline model.

Feature crossing v LR

To capture feature interactions better, we can add feature crosses to the input feature set.

What is feature crossing?

Feature crossing is a technique used in ML to create new features from existing features. It involves combining two or more features to capture the interaction between them. The idea is that by combining features, the model can learn more complex interactions between the original features in the data, which can improve the performance of ML models. For example, interactions such as "young and basketball" or "USA and football" may positively impact a model's ability to predict clicks probability.

How to create feature crosses?

In feature crossing, we manually add new features to the existing features based on prior knowledge. As Figure 4 shows, common feature crosses can be "age and city" or "language" pairs, as these features are meaningfully dependent. To learn more about crossing, refer to [4].

Figure 4: Creating feature crosses from two categorical features.

How to use feature crossing in LR?

As shown in Figure 5.2, feature crossing + LR works as follows:

- Use feature crossing on the original set of features to extract new features (crossed features).
- Use the original and the crossed features as input for the LR model.

Figure 5: Feature crossing.

Figure 5.1: Feature crossing + LR.

This method allows the model to capture complex and non-linear relationships between features. It has three disadvantages.

- **Manual process:** Human involvement is required to choose features to be crossed, which is time-consuming and expensive.
- **Requires domain knowledge:** Feature crossing requires domain expertise. To determine which interactions between features are meaningfully dependent for the model, we need to understand the problem and the feature space in detail.
- **Limited to linear models:** Feature crossed features can not capture non-linear relationships. To address these issues, we can use factorization machines (FM).

Factorization machines (FM)

FMs automatically learn feature interactions by representing each feature with a low-dimensional latent vector. FMs model feature interactions by calculating the dot product of their embeddings. Let's look at the equation and understand it step-by-step.

Figure 6.1: Simple embedding.

Figure 6.2: Simple embedding.

Two latent architectures: In this option, we use two encoders: user encoder and ad encoder. The similarity between ad and user embeddings is used to determine the relevancy. This is a two-tower model, which is covered in Chapter 2 Event Recommendations.

Figure 6.3: Two-tower architecture.

Although Wide Kaiss works beautifully, they may not be the best choice for ad click prediction systems because:

- **Sparsity:** Given that the feature space is usually huge and sparse, wide linear models and Wide&Deep models can still memorize sparse feature interactions, but they can't generalize to unseen feature combinations.
- **Difficult to capture all pairwise feature interactions:** due to the huge number of features.

Given these limitations, we will use DeepFM.

Deep & Cross Network (DCN)

Deep & Cross Network (DCN) was proposed by Google researchers in 2017 to address the feature interaction sparsity. This addresses the challenges of the manual feature crossing technique. The architecture is shown below. It consists of two components:

- **Deep network:** Learns complex and generalizable features using a Deep Neural Network (DNN) architecture.
- **Cross network:** Automatically creates explicit feature interactions and learns their importance.

The output of deep network and cross network are concatenated to make a final prediction.

There are two types of DCN architectures: stacked and parallel. Figure 8.2 shows the architecture of parallel DCN. To learn more about stacked architectures, refer to [7]. Note that it is not usually expected to provide details of DCN during ML system design interviews. If you are interested in learning more about DCN variations, refer to [7, 8].

Figure 8.1: Simple embedding.

Figure 8.2: DCN architecture.

Deep Factorization Machines (DeepFM)

DCN is a promising approach that combines explicit feature interactions learned from the cross network module. However, the cross network only models certain feature interactions, which may negatively affect the performance of the cross network module.

Factorization Machines (FM)

FMs automatically learn feature interactions by representing each feature with a low-dimensional latent vector. FMs model feature interactions by calculating the dot product of their embeddings. Let's look at the equation and understand it step-by-step.

Figure 8.3: DCN architecture.

$$\hat{y}(x) = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle v_i, v_j \rangle x_i x_j$$

Where x_i refers to the i-th feature, w_i is the learned weight, and v_i represents the embedding of the i-th feature. $\langle v_i, v_j \rangle$ denotes the dot product between two embeddings.

This formula may look complex, but it's actually easy to understand. The first two terms compute a linear combination of the features, similar to how logistic regression works. The third term models pairwise feature interactions. Figure 8.13 shows a high-level overview of FM. Refer to [9] to learn the details of FM.

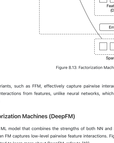

Figure 8.13: Factorization Machine architecture

FM and its variants, such as IFFM, effectively capture pairwise interactions between features. FM cannot learn sophisticated higher-order interactions from features, unlike neural networks, which can. In the next method, we combine FM and DNN to overcome this.

Deep Factorization Machines (DeepFM)

DeepFM is an ML model that combines the strengths of both NN and FM. A DNN network captures sophisticated higher-order features, and an FM captures low-level pairwise feature interactions. Figure 8.14 shows the high-level architecture of DeepFM. If you are interested to learn more about DeepFM, refer to [10].

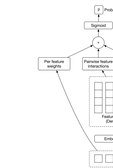

Figure 8.14: DeepFM overview

One potential improvement is to combine GBDT and DeepFM. A GBDT converts the original features into more predictive features, while DeepFM operates on the new features. This method has won various ad prediction system competitions [11]. However, adding GBDT to DeepFM negatively affects the training and inference speed, and slows down the continual learning process.

In practice, we usually choose the correct model by running experiments. In our case, we start with a simple LR to create a baseline. Next, we experiment with DCN and DeepFM, as both are widely used in the tech industry.

Model training

Constructing the dataset

For every ad impression, we construct a new data point. The input features are computed from the user and the ad. A label is assigned to the data point, based on the following strategy:

- **Positive label:** If the user clicks the ad in less than t seconds after the ad is shown, we label the data point as "positive". Note that t is a hyperparameter and can be tuned via experimentation.
- **Negative label:** If the user does not click the ad in less than t seconds, we label the data point as "negative".

In practice, companies use more complex methods to find the optimal strategy for labeling negative data points. To learn more, refer to [3].

#	User and interaction features					Ad features					Label
1	1	0	1	0.8	0.5	1	0	0	1	1	Positive
2	1	1	0	0.6	0.9	1	1	1	0	0.7	Negative

Figure 8.15: Constructed dataset

To keep the model adaptive to new data, it must continuously be trained. As a result, new training data points should be continuously generated using new interactions. We will discuss continual learning further in the serving section.

Choosing the loss function

Since we are training a binary classification model, we choose cross-entropy as a classification loss function.

Evaluation

Offline metrics

Two metrics are typically used to evaluate an ad click prediction system:

- Cross-entropy (CE)
- Normalized cross-entropy (NCE)

CE. This metric measures how close the model's predicted probabilities are to the ground truth labels. CE is zero if we have an ideal system that predicts a 0 for the negative classes and 1 for the positive classes. The lower the CE, the higher the accuracy of the prediction. The formula is:

$$H(p, q) = -\sum_{i=1}^{n} p_i \log q_i$$

where p_i is the ground truth, q_i is the predicted probability, and C is the total number of classes.

For binary classification, the CE formula can be rewritten as:

$$H(p, q) = -\sum_{i=1}^{n} y_i \log q_i = -\sum_{i=1}^{n} (y_i \log q_i + (1 - y_i) \log (1 - q_i))$$

where y_i is the ground truth label of the i-th data point and $\hat{y}_i$ is the predicted probability of i-th data point.

Let's take a look at a concrete example, as shown in Figure 8.16.

Figure 8.16: CE of two ML models

Note that aside from using CE as a metric, it is also commonly used as a standard loss function in classification tasks during model training.

Normalized cross-entropy (NCE). NCE is the ratio between our model's CE and the CE of the background CTR (average CTR in the training data). In other words, NCE compares the model with a simple baseline which always predicts the background CTR. A low NCE indicates the model outperforms the simple baseline. NCE of 1 indicates that the model is not performing better than the simple baseline.

$$\text{Normalized cross entropy} = \frac{\text{CE of ML model}}{\text{CE (Simple baseline)}}$$

Let's take a look at a concrete example to understand better how NCE is calculated. As shown in Figure 8.17, a simple baseline model always predicts 0.6 (CTR in the training data). In this case, the NCE value is 0.324 (less than 1), indicating model A outperforms the simple baseline.

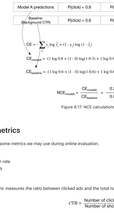

Figure 8.17: NCE calculations for model A

Online metrics

Let's examine some metrics we may use during online evaluation.

- CTR
- Conversion rate
- Revenue lift
- Hide rate

CTR. This metric measures the ratio between clicked ads and the total number of shown ads.

$$\text{CTR} = \frac{\text{Number of clicked ads}}{\text{Number of shown ads}}$$

CTR is a great online metric for ad click prediction systems, as maximizing user clicks on ads is directly related to an increase in revenue.

Conversion rate. This metric measures the ratio between the number of conversions and the total number of ads shown.

$$\text{Conversion rate} = \frac{\text{Number of conversions}}{\text{Number of impressions}}$$

This metric is important to track as it indicates how many times advertisers actually benefited from the system. This matters because advertisers will eventually lose interest and cease spending on ads if their ads do not lead to conversions.

Revenue lift. This measures the percentage of revenue increase over time.

Hide rate. This metric measures the ratio between the number of ads hidden by users and the number of shown ads.

$$\text{Hide rate} = \frac{\text{Number of ads hidden by users}}{\text{Number of shown ads}}$$

This metric is helpful for understanding how many irrelevant ads the system displayed to users, also known as false positives.

Serving

At serving time, the system is responsible for outputting a list of ads ranked by their click probabilities. The proposed ML system design is shown in Figure 8.18. Let's examine each of the following pipelines:

- Data preparation pipeline
- Continual learning pipeline
- Prediction pipeline

Figure 8.18: ML system design

Data preparation pipeline

The data preparation pipeline performs the following two tasks:

1. Compute online and batch features
2. Continuously generate training data from new ads and interactions

To compute features, the following two options are used: batch feature computation and online feature computation. Let's see how they differ from each other.

Batch feature computation Some of the features we chose are static, which means they change very rarely. For example, an ad's image and category are static features. This component computes static features periodically (e.g., every few days or weeks) with batch jobs and then stores the features in a feature store. This improves the system's performance during serving because the features are precomputed.

Online feature computation Some features are dynamic as they change frequently. For example, the numbers of ad impressions and clicks are examples of dynamic features. These features need to be computed at query time, and this component is used to compute dynamic features.

Continual learning pipeline

Based on the requirements, continually learning the model is critical. This pipeline is responsible for fine-tuning the model on new training data, evaluating the new model, and deploying the model if it improves the metrics. It ensures the prediction pipeline always uses a model adapted to the most recent data.

Prediction pipeline

The prediction pipeline takes a query user as input and outputs a list of ads ranked by their click probabilities. Since some of the features which the model relies upon are dynamic, we cannot use batch prediction. Instead, requests are served as they arrive using online prediction.

As we've seen in previous chapters, a two-stage architecture is used in the prediction pipeline. First, we employ a candidate generation service to efficiently narrow down the available pool of ads to a small subset of ads. In this case, we use the ad targeting criteria other provided by advertisers, such as target age, gender, and country.

Next, we employ a ranking model which fetches the candidate ads from the candidate generation service, ranks them based on click probability, and outputs the top ads. This component interacts with the same feature store and online feature computation component. Once the static and dynamic features are obtained, the ranking service uses the model to get a predicted click probability for each candidate ad. These probabilities are used to rank the ads and to output those with the highest click probability.

Finally, a re-ranking service modifies the list of ads by incorporating additional logic and heuristics. For example, we can increase the diversity of ads by removing very similar ads from the list.

Other Talking Points

If there is time left at the end of the interview, here are some potential talking points you might discuss with the interviewer:

- In ranking and recommendation systems, it's important to avoid data leakage [12][13].
- The model needs to be calibrated in ad click prediction systems. Discuss model calibration and techniques for calibrating a model [14].
- A common variant of FM is a field-aware Factorization Machine (FFM). It's good to talk about FFM and how it differs from FM [15].
- A common variant of DeepFM is XDeepFM. Talk about how XDeepFM and how it differs from DeepFM [16].
- We've described why continuous learning is necessary for ad click prediction systems. However, continual learning on new data may lead to catastrophic forgetting. Discuss what catastrophic forgetting is and what common solutions are [16].

References

1. Addressing delayed feedback. https://arxiv.org/abs/1907.06558.pdf
2. AdTech terms: bids.//advertising.amazon.com/library/guides/adtech/ip-adtech.
3. SmCLR paper. https://arxiv.org/pdf/2002.05709.pdf.
4. Feature crossing. https://developers.google.com/machine-learning/crash-course/feature-crosses/video-lecture
5. Feature extraction with GBDT. https://towardsdatascience.com/gradient-boosted-decision-trees-explained-9259bd8205af.
6. DCN paper. https://arxiv.org/pdf/1708.05123.pdf
7. DCN V2 paper. https://arxiv.org/pdf/2008.13535.pdf
8. Microsoft's deep crossing network paper. https://www.bing.com/cr/2016/2017/Factorization-machines/
9. Factorization Machines. https://www.jeffkreng.com/recsys/2017/2017/Factorization-machines/
10. Deep Factorization Machines. https://ifi.uzh.ch/dam/jcr:ffffffff-ffff-ffff-0000-000000000000/kons/lewis.html
11. Kaggle's winning solution in ad click prediction. https://www.youtube.com/watch?v=Ko2GcvRYvuU.
12. Data leakage in ML systems. https://machinelearningmastery.com/data-leakage-machine-learning/
13. Time-based dataset splitting https://www.linkedin.com/pulse/time-based-splitting-determining-train-test-data-come-marwa-chaabouni/?trk=public_profile_article_view
14. Model calibration. https://machinelearningmastery.com/calibration-classification-model-in-scikit-learn/
15. Field-aware Factorization Machines. https://www.csie.ntu.edu.tw/~cjlin/papers/ffm.pdf.
16. Catastrophic forgetting problem in continual learning. https://news.sv.cmu.edu/-/subfitness-learning/continual-learning.pdf.

Mark as Complete

Partner With Us
Go an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
help@educative.io
Report a Bug

Resources
YouTube
Visual Dev Guides
Blog
Find Jobs
Prepare for Coding Interviews

Copyright ©2022-2026 BytByteGo Inc. All rights reserved.